HLD Level 2.3

Design a Real-Time Camera / Video Processing Pipeline — Complete Interview Question & Answer

> **Interview question: Design an edge camera system that captures video, performs real-time object detection, provides a live stream, records video, and remains low-latency and stable when processing temporarily falls behind the camera frame rate.**

1. Requirements

Functional: camera capture, real-time object detection, live video streaming, recording, detection events/metadata, device health and telemetry.

Non-functional: target ~30 FPS, low detection latency, fresh live video, bounded memory, long-running stability, failure recovery, and efficient CPU/accelerator use.

> **Critical requirement: for live detection and live viewing, freshness is usually more important than processing every frame. Recording can have different completeness/durability requirements.**

2. Capacity Estimation

```
1920 x 1080 RGB
3 bytes/pixel
~6.2 MB/raw frame

30 FPS -> ~186 MB/sec raw frame data
70 retained raw frames -> ~434 MB payload
```

This immediately makes frame copies, queue sizes, buffer ownership and encoding important design concerns. Real systems may use lower-resolution detection streams and encoded video to reduce bandwidth/storage.

3. Top-Level Architecture

```
Camera Sensor
|
Capture Layer
|
+--------------+--------------+
| | |
v v v
Detection Live Stream Recording
|
Preprocess
|
Inference
|
Postprocess
|
Events / Metadata
```

4. Why Separate Pipelines?

Detection, streaming and recording have different latency/durability goals. A slow inference stage should not automatically block streaming and recording.

```
Bad: Capture -> Inference -> Stream -> Record

Better: Capture -> Detection
-> Streaming
-> Recording
```

5. Detection Pipeline

```
Capture -> Bounded Frame Queue -> Preprocess -> Inference -> Postprocess -> Detection Event
```

Preprocessing can include resize, crop, padding, color/format conversion and normalization. Inference may run on CPU/GPU/NPU/AI accelerator. Postprocessing can include confidence filtering, NMS, ROI checks, bounding boxes and event generation.

6. Producer-Consumer Rate Mismatch

```
Camera producer = 30 FPS
Inference consumer = 20 FPS
Backlog growth = +10 frames/sec
After 10 sec = 100 stale frames
```

For a real-time detector, processing old frames seconds later is often worse than intentionally dropping stale frames.

7. Bounded Queue

Use a small bounded queue between capture and detection. The bound limits both memory and queueing delay.

> **Queue capacity is not only a memory decision; in a real-time pipeline it is also a latency decision.**

8. Drop-Oldest Policy

```
Full queue: F100 F101 F102 F103 F104
New frame: F105

Drop F100 -> F101 F102 F103 F104 F105
```

Keeping the newest frame preserves freshness. This policy is appropriate only when the product values current state more than processing every historical frame.

9. Freshness vs Completeness

```
Detection -> freshness
Live stream -> freshness
Recording -> completeness/durability
```

Do not blindly apply one queue-full policy to every branch. Recording may need stronger buffering/storage handling rather than drop-oldest.

10. Latency Budget

```
Capture 5 ms
Queue wait 2 ms
Preprocess 8 ms
Inference 25 ms
Postprocess 5 ms
------------------
Total 45 ms
```

Stage-level timing lets us locate the bottleneck. End-to-end latency alone cannot tell whether the problem is inference, queueing, preprocessing or I/O.

11. Queueing Latency

```
Inference execution = 30 ms
Queue wait = 150 ms
Total = 180 ms
```

Optimizing inference from 30 to 25 ms barely helps if queue wait dominates. Rising queue wait/depth is often the earliest sign that the consumer is not keeping up.

12. Concurrency Model

Possible threads include capture, preprocessing, inference/postprocessing, streaming and recording. Exact thread boundaries depend on hardware APIs and measured bottlenecks; do not mechanically create one thread for every conceptual box.

13. Stage Queues

```
Capture -> Q1 -> Preprocess -> Q2 -> Inference -> Q3 -> Postprocess
```

Stage queues enable pipelining and independent timing, but every queue adds memory, synchronization and latency. Keep queues intentionally small and justified.

14. C++ Frame Ownership

Multi-megabyte frame copies are expensive. Prefer explicit ownership transfer, move semantics, reusable buffers, unique ownership where possible, and carefully justified shared ownership.

```
Bad: Capture -> COPY -> Preprocess -> COPY -> Inference

Better: Capture -> buffer ownership -> move/reference -> stages
```

15. Buffer Pool

```
Reusable pool: B1 B2 B3 B4 B5

Capture gets B1 -> pipeline uses B1 -> B1 returned to pool
```

Buffer pools reduce repeated large allocations/deallocations, improve predictability and can reduce fragmentation. Pool size itself must be bounded.

16. Hardware Accelerator

```
CPU preprocess -> accelerator input -> inference -> CPU postprocess
```

Consider accelerator queue depth, input format, DMA/copy cost, synchronization and recovery. Large batching may improve throughput but increase latency; real-time camera inference often favors small batches or batch=1.

17. Live Streaming

```
Capture -> Encoder -> Stream Server -> Network -> Viewer
```

Raw RGB bandwidth is too high for normal remote viewing, so video is encoded (for example H.264/H.265). A slow viewer must not cause unlimited buffering.

18. Slow Viewer

If a camera produces 30 FPS but the viewer/network can effectively consume only 10 FPS, accumulating 20 FPS forever creates stale video. Possible responses include dropping stale frames, reducing bitrate/resolution/FPS or adaptive streaming.

19. Recording Pipeline

```
Capture -> Encoder -> Recording Queue -> Storage Writer -> Disk/Object Storage
```

Recording has different semantics. Monitor disk latency, queue depth, free space and write throughput. Depending on requirements, rotate old recordings, degrade quality, or raise health alarms rather than blindly using the detection drop policy.

20. Detection Events

```
Inference -> Postprocess -> Event Queue -> Network Publisher
```

Detection metadata should normally leave the inference hot path asynchronously. Events can include timestamp, object type, confidence, ROI, frame/event ID and device metadata.

21. Control and Telemetry Paths

```
Device -> heartbeat
-> telemetry
-> detection events
<- configuration
<- commands
```

Control/network activity should be isolated so temporary network problems do not unnecessarily block local capture and inference.

22. Health Metrics

Track capture FPS, inference FPS, per-stage latency, queue depth/wait, dropped frames, P50/P95/P99 end-to-end latency, CPU, memory, temperature, accelerator utilization, disk utilization/free space and network health.

23. Detecting Saturation

```
CPU = 95%
Detection queue = 4/5
Dropped frames rising
P99 latency = 200 ms
```

This suggests the device is near or beyond its sustainable operating point. Responses can include frame skipping, lower inference resolution/FPS, optimization, accelerator offload, stream quality reduction or disabling noncritical work.

24. Why Not Increase Queue Size?

Increasing a detection queue from 5 to 100 frames increases buffering and may make detections stale. A larger queue can improve burst absorption but can violate the latency/freshness objective.

> **For real-time systems, more buffering can make the product worse even when it reduces drops.**

25. Failure Handling

Capture failure: restart/reinitialize capture and emit health signal.
Accelerator failure: reset/reinitialize or degrade/fallback if supported.
Network failure: keep local detection/recording functioning where possible and bound outbound event buffering.
Storage pressure: rotate/degrade/alert according to recording requirements.

26. Process Supervision and RAII

A supervisor such as systemd can restart a crashed service. Inside the C++ process, RAII should manage camera handles, sockets, file descriptors, buffers, threads and accelerator resources so cleanup remains deterministic.

27. Graceful Shutdown

```
SIGTERM
-> stop accepting/capturing new work
-> signal pipeline stop
-> wake condition variables
-> drain/cancel queues by policy
-> finish required storage writes
-> join threads
-> release camera/accelerator/resources
-> exit
```

28. Performance Profiling

Use std::chrono::steady_clock for elapsed stage durations. Track distributions rather than only averages.

```
Capture 5ms
Preprocess 8ms
Inference 30ms
Postprocess 5ms

Track P50 / P95 / P99 / worst-case
```

29. Stable Operating Point

```
30 FPS:
CPU 70% | queue 1-2 | P99 55ms | stable

45 FPS:
CPU 98% | queue full | drops high | P99 300ms | occasional instability
```

Selecting 30 FPS can be the correct engineering decision because it preserves headroom and tail-latency stability. Maximum benchmark throughput is not necessarily the correct production operating point.

30. Complete Architecture

```
CAMERA SENSOR
|
v
Capture Service
|
+-----------------+-----------------+
| | |
v v v
Detection Queue Stream Queue Recording Queue
(small/bounded, (bounded, (bounded,
freshness) freshness) durability)
| | |
v v v
Preprocess Encoder Encoder
| | |
v v v
Inference Stream Server Storage Writer
Accelerator | |
| v v
v Viewer Disk
Postprocess
|
v
Detection Events
|
v
Event Queue -> Network Publisher

+ Heartbeat / Health / Telemetry / Config
```

31. Common Interview Follow-Ups

30 FPS capture but 20 FPS inference? Small bounded queue; drop stale frames if freshness is the requirement.

Why not queue 1000 frames? Memory and stale-latency explosion.

Why separate recording? Recording completeness differs from live detection freshness.

How reduce memory/copy cost? Buffer pools, explicit ownership, move semantics, avoid unnecessary conversions/copies.

How detect bottleneck? Stage timings + queue wait/depth + CPU/accelerator/disk/network metrics.

Why not always maximize FPS? Sustainable headroom and tail latency matter more than unstable peak throughput.

32. Interview-Ready Spoken Answer

"I would separate capture, detection, live-streaming and recording because they have different latency and durability requirements. Capture feeds bounded queues. For real-time detection, freshness matters more than processing every frame, so if inference falls behind I would keep the queue small and drop stale frames instead of allowing latency to grow indefinitely. Detection consists of preprocessing, hardware-accelerated inference and postprocessing, with stage-level timing and queue-wait metrics. Streaming uses encoded video and also prioritizes freshness, while recording has stronger completeness requirements and therefore uses a different buffering/storage policy. In C++, I would minimize large frame copies using explicit ownership and reusable buffer pools, use condition variables rather than busy waiting, bound every queue, isolate network work from the frame-processing hot path, and use RAII with graceful shutdown. If producer rate permanently exceeds processing capacity, I would not solve it by simply increasing queue size; I would reduce work, skip frames, optimize the pipeline, use hardware acceleration or lower the operating FPS."

33. Five Decisions to Remember

1. Different requirements -> SEPARATE DETECTION / STREAM / RECORDING PIPELINES.
2. Real-time detection -> FRESHNESS > PROCESSING EVERY FRAME.
3. Slow consumer -> SMALL BOUNDED QUEUE + EXPLICIT DROP POLICY.
4. Large C++ frames -> MINIMIZE COPIES + BUFFER POOL + CLEAR OWNERSHIP.
5. Performance issue -> MEASURE STAGES + QUEUE WAIT + P95/P99 BEFORE OPTIMIZING.

34. Final Revision Flow

```
Requirements
-> capacity/frame memory
-> split pipelines
-> producer/consumer
-> bounded queues/drop policy
-> latency budget
-> ownership/buffer pool
-> accelerator
-> streaming vs recording
-> failures
-> observability/profiling
-> graceful shutdown
-> stable operating point
```